

## Table of contents

|                                                                   |          |
|-------------------------------------------------------------------|----------|
| <b>Série 7 : DPLL avec théories et théorie des tableaux</b>       | <b>1</b> |
| Méthodes . . . . .                                                | 1        |
| DPLL(T) . . . . .                                                 | 1        |
| Théorie des entiers linéaires . . . . .                           | 1        |
| Théorie EUF (Equality with Uninterpreted Functions) . . . . .     | 1        |
| Théorie des tableaux . . . . .                                    | 2        |
| Formalisation en logique du premier ordre . . . . .               | 2        |
| Vérification d'équivalence de code . . . . .                      | 2        |
| Rappel . . . . .                                                  | 2        |
| Exemples Illustratifs . . . . .                                   | 2        |
| Exemple 1 : DPLL avec la théorie des entiers . . . . .            | 2        |
| Exemple 2 : Théorie des tableaux et équivalence de code . . . . . | 3        |
| Synthèse . . . . .                                                | 4        |

## Série 7 : DPLL avec théories et théorie des tableaux

### Méthodes

#### DPLL(T)

Méthode de décision pour les formules du premier ordre combinant l'algorithme DPLL classique (pour la partie propositionnelle) avec un solveur de théorie (T) pour vérifier la cohérence des littéraux dans une théorie spécifique (ex : entiers, EUF, tableaux).

Sous-catégories :

- **DPLL propositionnel** : attribution de valeurs de vérité, propagation unitaire, backtrack.
- **Intégration théorie** : le solveur de théorie valide ou invalide les affectations partielles en fonction des contraintes de la théorie.

#### Théorie des entiers linéaires

Théorie permettant de raisonner sur les inégalités et égalités linéaires entre variables entières. Utilisée ici pour interpréter des littéraux comme  $a + 1 \leq b$ .

#### Théorie EUF (Equality with Uninterpreted Functions)

Théorie de l'égalité étendue à des symboles fonctionnels non interprétés. Les axiomes sont la réflexivité, la symétrie, la transitivité et la congruence : si  $x = y$  alors  $f(x) = f(y)$ .

## Théorie des tableaux

Théorie spécifique aux opérations de lecture (**select**) et écriture (**store**) sur des tableaux.  
Axiomes fondamentaux :

1. Lecture après écriture au même indice.
2. Lecture après écriture à un indice différent.
3. Extensionnalité : deux tableaux égaux si tous leurs éléments sont égaux.

## Formalisation en logique du premier ordre

Traduction d'énoncés informels (propriétés de programmes, de structures de données) en formules quantifiées de la logique du premier ordre, utilisables ensuite par des solveurs automatisés.

## Vérification d'équivalence de code

Comparaison de deux fragments de code en exprimant leur sémantique à l'aide d'une théorie (ex : tableaux) et recherche d'une condition logique garantissant l'égalité des résultats.

## Rappel

- **Algorithme DPLL** : recherche récursive d'un modèle pour une formule propositionnelle, avec propagation unitaire et backtrack.
- **Axiomes des tableaux** :
  - $\forall a, i, e. \text{select}(\text{store}(a, i, e), i) = e$
  - $\forall a, i, j, e. i \neq j \Rightarrow \text{select}(\text{store}(a, i, e), j) = \text{select}(a, j)$
  - $\forall a, b. (\forall i. \text{select}(a, i) = \text{select}(b, i)) \Rightarrow a = b$
- **Congruence en EUF** :  $\forall x, y. x = y \Rightarrow f(x) = f(y)$
- **Condition d'équivalence de deux stores successifs** :  
 $\text{store}(\text{store}(a, i, b), j, c) = \text{store}(\text{store}(a, j, c), i, b)$  ssi  $(i \neq j) \vee (i = j \wedge b = c)$

## Exemples Illustratifs

### Exemple 1 : DPLL avec la théorie des entiers

Contexte :

On a une formule  $\phi = (p \vee q) \wedge (\neg p \vee r) \wedge (s \vee t) \wedge (\neg r \vee s \vee u)$ , où chaque variable propositionnelle est associée à une contrainte sur les entiers (ex :  $p$  correspond à  $a + 1 \leq b$ ). On cherche une affectation satisfaisant  $\phi$  et les contraintes de la théorie.

### Application :

1. **DPLL propositionnel** : on part de l'affectation  $p \leftarrow \text{True}$ .

- $\phi[p] = r \wedge (s \vee t) \wedge (\neg r \vee s \vee u)$ .
- On choisit  $r \leftarrow \text{True}$  :  $\phi[p, r] = (s \vee t) \wedge (s \vee u)$ .
- On choisit  $s \leftarrow \text{True}$  :  $\phi[p, r, s] = \text{True}$ .
- Modèle propositionnel :  $I = \{p \leftarrow \text{True}, r \leftarrow \text{True}, s \leftarrow \text{True}\}$ .

2. **Vérification théorie des entiers** :

Les contraintes correspondantes sont :

- $p : a + 1 \leq b$
- $r : a > b + c$
- $s : b > 0$

On doit trouver des entiers  $a, b, c$  satisfaisant ces trois inégalités.

Choix :  $a = 10, b = 11, c = -2$

- $10 + 1 \leq 11$  vraie,
- $10 > 11 + (-2)$  soit  $10 > 9$  vraie,
- $11 > 0$  vraie.

Donc le modèle est valide dans la théorie des entiers.

### Exemple 2 : Théorie des tableaux et équivalence de code

#### Contexte :

On compare deux séquences d'écritures dans un tableau :

- `array1 = store(store(a,i,b), j, c)`
- `array2 = store(store(a,j,c), i, b)`

On veut savoir quand `array1` égale `array2`.

#### Application : 1. Cas $i \neq j$ :

Les écritures sont à des indices distincts, l'ordre n'a pas d'importance. Les deux séquences produisent un tableau où :

- à l'indice  $i$  : valeur  $b$ ,

- à l'indice  $j$  : valeur  $c$ ,
- ailleurs identique à  $a$ .

Donc égalité.

2. **Cas  $i = j$  :**

Les deux écritures sont au même indice. La dernière écriture écrase la première.

- `array1` met d'abord  $b$  puis  $c$  : résultat final  $c$  à l'indice  $i$ .
- `array2` met d'abord  $c$  puis  $b$  : résultat final  $b$  à l'indice  $i$ .

Pour l'égalité, il faut  $b = c$ .

3. **Condition logique :**

La condition d'équivalence est donc :

$$(i \neq j) \vee (i = j \wedge b = c)$$

Ce qui peut s'écrire en FOL avec la théorie des tableaux et l'égalité.

4. **Vérification Why3 :**

On peut encoder cette condition comme une précondition dans Why3 pour vérifier que les deux tableaux sont égaux. Si la condition est respectée, le vérificateur affichera "green V".

## Synthèse

Cette série illustre l'utilisation combinée de méthodes de satisfaction logique (DPLL) avec des théories du premier ordre (entiers, EUF, tableaux) pour modéliser et vérifier des propriétés de programmes. La clé est de séparer la partie propositionnelle de la partie théorie, et d'utiliser des solveurs spécialisés pour chaque théorie. La formalisation en logique du premier ordre et l'utilisation d'outils comme Why3 permettent d'automatiser la vérification de conditions d'équivalence ou de validité.